DEVOPS INTERNSHIP PROJECT REPORT

End-to-End CI/CD Pipeline for a Java Web Application

Git · Maven · Jenkins · Apache Tomcat

Application: Student Feedback Portal

Submitted by	Ayushi Tripathi
Environment	Kali Linux (WSL2) on Windows
Source control	Git + GitHub
CI server	Jenkins (port 8085)
Application server	Apache Tomcat 10.1.34 (port 8081)
Date	October 2026

Table of Contents

1. Introduction and Problem Statement 3

2. Objectives and Scope 3

3. Tools and Technologies 4

4. DevOps Concepts Used 5

5. System Architecture and Application Design 6

6. Implementation 8

7. Deployment Verification 15

8. Testing and Test Cases 17

9. Failure Handling and Rollback 18

10. Challenges and Solutions 21

11. Risks and Mitigation 22

12. Results: Manual vs Automated Delivery 22

13. Limitations and Future Improvements 23

14. Deliverables Checklist 23

15. Conclusion 23

Appendix A: Command Reference 25

Appendix B: Evidence Index 25

1. Introduction and Problem Statement

Software delivery in many student and early-stage projects is still manual. A developer writes code, builds it locally with Maven, copies the WAR file by hand, deploys it to Tomcat and then checks the site in a browser. This is slow and error-prone. Typical failures are:

- the wrong artifact version is deployed;
- unit tests are skipped before release;
- the developer machine and the server behave differently;
- there are no clear logs to debug a failed release;
- there is no rollback plan when a deployment breaks the application.

DevOps addresses this by automating the path from a code commit to a running application and by adding verification and recovery steps. This project implements that idea end to end for a Java web application called the **Student Feedback Portal**.

1.1 What the pipeline does

A change committed to Git is picked up by a Jenkins job, which clones the project, builds and unit-tests it with Maven, produces `app.war`, and copies it into Apache Tomcat. The deployed application is then verified through a health endpoint (`/app/health`). Failure and rollback cases were also demonstrated so that the process is shown to be safe, not just fast.

2. Objectives and Scope

2.1 Objectives

1. Design a workflow that follows the SDLC stages: plan, code, build, test, release, deploy and monitor.
2. Maintain source code in Git with a clear branching strategy (`main`, `dev`, `feature/form`) and version tags.
3. Build and package a Java web application into a WAR using Apache Maven.
4. Automate build, test and deployment with a Jenkins job.
5. Deploy the generated WAR file automatically to Apache Tomcat.
6. Verify every deployment with a health check.
7. Demonstrate a failing-test quality gate and a rollback to the previous working WAR.
8. Document the work with logs and screenshots as evidence.

2.2 Scope

In scope	Out of scope
One Java web application (Maven WAR project) Git repository with branches, tags and commit conventions Unit test executed on every build Jenkins job for build, test and deploy Automated deployment to Tomcat Health-check verification Failure and rollback demonstration	Kubernetes or Docker Swarm clusters Multi-region deployments Enterprise security scanners (SAST/DAST) Blue-green or canary deployment at scale

Dataset: not required; the primary project input is source-code changes.

3. Tools and Technologies

Component	Tool / Version	Role in this project
Operating system	Kali Linux on WSL2	Linux environment for all tools and shell commands.
Language	Java (OpenJDK 21)	Compiles the servlets; Tomcat 10.1 runs on this JRE.
Build tool	Apache Maven 3.9.12	Compile, test and package the WAR with `mvn clean package`.
Version control	Git 2.53.0 + GitHub	Source history, branches, tags and the remote repository.
CI server	Jenkins (port 8085)	Runs the job `devops-ci-cd-pipeline`: clone, build, test, deploy.
Application server	Apache Tomcat 10.1.34 (port 8081)	Hosts the WAR under the context path `/app`.
Artifact	`app.war`	Deployable Java Web Archive produced by Maven.
Helper tools	`curl`, `cp`, `sudo`	Health checks, copying the WAR, privileged actions.

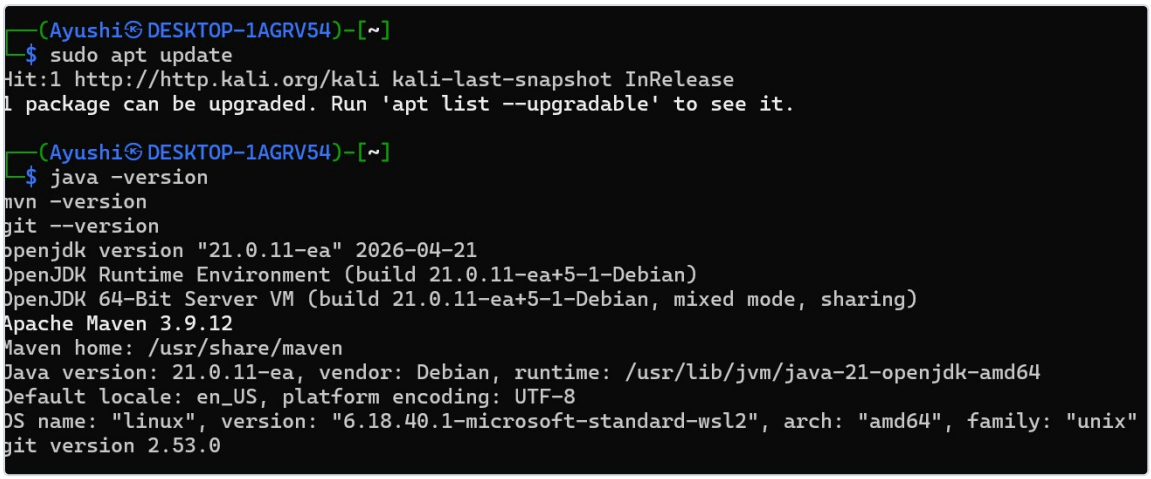


Figure 1: Environment verification: Java, Maven and Git versions on Kali Linux (WSL2).

4. DevOps Concepts Used

4.1 SDLC and DevOps lifecycle mapping

Stage	What was done in this project	Tool
Plan	Defined the portal, its pages and the health endpoint	-
Code	Wrote servlets, page and unit test; branching and commits	Git, GitHub
Build	Compiled and packaged into `app.war`	Maven
Test	JUnit test `StatusTest` run on every build	Maven Surefire
Release	Merged to `main`, tagged `v1.0` and `v1.1`	Git
Deploy	WAR copied into Tomcat `webapps`	Jenkins, Tomcat
Monitor	Health endpoint and Tomcat/Jenkins logs	`curl`, browser

4.2 Continuous Integration and Continuous Deployment

Continuous Integration (CI) means every change is automatically built and tested so that errors are found early. Here Jenkins clones the repository and runs Maven, whose test phase acts as the gate. **Continuous Deployment (CD)** means a build that passes is released automatically. Here the same Jenkins job copies the tested WAR into Tomcat without any manual step.

4.3 WAR packaging and Tomcat

A WAR (Web Application Archive) bundles the compiled classes, pages and configuration of a Java web application in one file. When `app.war` is placed in Tomcat's `webapps` directory, Tomcat unpacks

it and serves it under the context path `/app`. This is why the application is available at `http://localhost:8081/app/`.

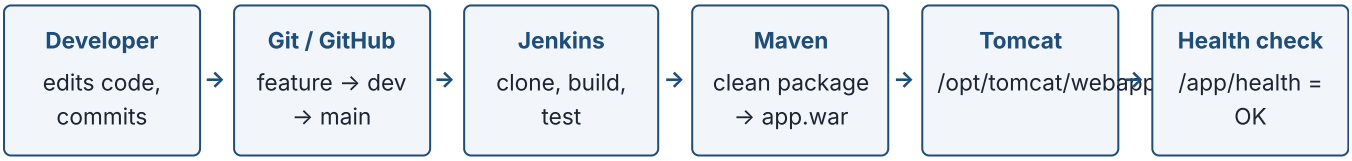
4.4 Maven build lifecycle

The command `mvn clean package` runs the following plugin goals, all visible in the Jenkins console logs of this project:

Plugin goal	Purpose
clean 3.2.0 : clean	Deletes the old `target/` directory so each build starts fresh.
resources 3.3.1 : resources	Copies main resources (none existed, so it is skipped).
compiler 3.13.0 : compile	Compiles the 3 main Java sources with target Java 21.
resources : testResources compiler : testCompile	Prepare and compile the unit test class.
surefire 3.2.5 : test	Runs `StatusTest` through the JUnit platform provider. A failure stops the build.
war 3.4.0 : war	Assembles the web application and builds `target/app.war`.

5. System Architecture and Application Design

5.1 Pipeline flow



> **Quality gates:** (1) if a unit test fails, Maven stops with BUILD FAILURE and nothing is copied to Tomcat; (2) if the WAR is not generated the build fails; (3) if the health endpoint does not return HTTP 200 after deployment, the previous known-good WAR is restored.

5.2 Port allocation

Service	Port	Note
Jenkins	8085	Changed from the default 8080 to avoid a conflict.
Apache Tomcat	8081	Application URL: `http://localhost:8081/app/`

5.3 Project structure

The repository `devops-ci-cd-project` contains a standard Maven web application layout:

```
devops-ci-cd-project/
|-- pom.xml                      Maven build file (packaging = war, final name = app)
|-- Jenkinsfile                  CI file kept in the repository
|-- .gitignore                   ignores target/ and other build output
`-- src/
    |-- main/
    |   |-- java/com/demo/
    |   |     FeedbackServlet.java    handles form submission
    |   |     HealthServlet.java      health endpoint
    |   |     Status.java             health message used by the unit test
    |   `-- webapp/index.html         feedback form page
    `-- test/java/com/demo/
          StatusTest.java             JUnit test healthMessageIsOk
```

5.4 Application endpoints

URL	Component	Behaviour
`/app/`	`index.html`	Feedback form with Name, Email and Feedback fields, a Submit button and a Health check link.
`/app/submit`	`FeedbackServlet`	Shows a confirmation page with the submitted name, email and feedback.
`/app/health`	`HealthServlet`	Returns plain text `OK` with HTTP 200; used by the pipeline and the rollback check.

6. Implementation

6.1 Git repository and branching

The repository was initialised with `main` as the stable branch. A `dev` branch serves as the integration branch and `feature/form` is used for feature work. The first commit added the whole Maven project (7 files, 116 insertions) and was tagged `v1.0`.

Branch	Purpose
`main`	Stable, deployable code. Releases are tagged here.
`dev`	Integration branch where finished features are merged first.
`feature/form`	Short-lived branch for the form-page change.

```
┌──(Ayushi㉿DESKTOP-1AGRV54)-[~/devops-ci-cd-project]
└─$ git config --global user.name "Ayushi"
git config --global user.email "triayushi0314@email.com"
git init -b main
printf "target/\n" > .gitignore
git add . && git commit -m "feat: add feedback portal with health endpoint"
git branch dev
git checkout -b feature/form
git checkout dev
git merge feature/form
git checkout main
git merge dev
git tag v1.0
git log --oneline --all
Initialized empty Git repository in /home/Ayushi/devops-ci-cd-project/.git/
[main (root-commit) 8f4bf69] feat: add feedback portal with health endpoint
 7 files changed, 116 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 pom.xml
 create mode 100644 src/main/java/com/demo/FeedbackServlet.java
 create mode 100644 src/main/java/com/demo/HealthServlet.java
 create mode 100644 src/main/java/com/demo/Status.java
 create mode 100644 src/main/webapp/index.html
 create mode 100644 src/test/java/com/demo/StatusTest.java
Switched to a new branch 'feature/form'
Switched to branch 'dev'
Already up to date.
Switched to branch 'main'
Already up to date.
8f4bf69 (HEAD -> main, tag: v1.0, feature/form, dev) feat: add feedback portal with health endpoint
```

Figure 2: Repository initialisation, branch creation, first commit and tag v1.0.

6.2 Commit convention

Commit messages follow a type prefix so the history explains itself:

Commit message	Type	Meaning
`feat: add feedback portal with health endpoint`	feat	Initial application (tag v1.0)
`feat: add footer message to form page`	feat	Footer text added to index.html (tag v1.1)
`chore: ignore deb files`	chore	Repository housekeeping in .gitignore
`ci: fix health check port to 8081`	ci	Health check aligned with the Tomcat port

6.3 Feature change, merge and release tag

A new change was made on `feature/form`: the line `<p>Thank you for your feedback!</p>` was appended to `src/main/webapp/index.html` and committed. The branch was then merged `feature/form → dev → main` (both fast-forward merges) and tagged `v1.1`. The page heading was also customised to **"Student Feedback Portal: BY Ayushi"**.

```
git checkout feature/form
git commit -am "feat: add footer message to form page"
git checkout dev   && git merge feature/form
git checkout main  && git merge dev
git tag v1.1
git log --oneline --graph --all
```

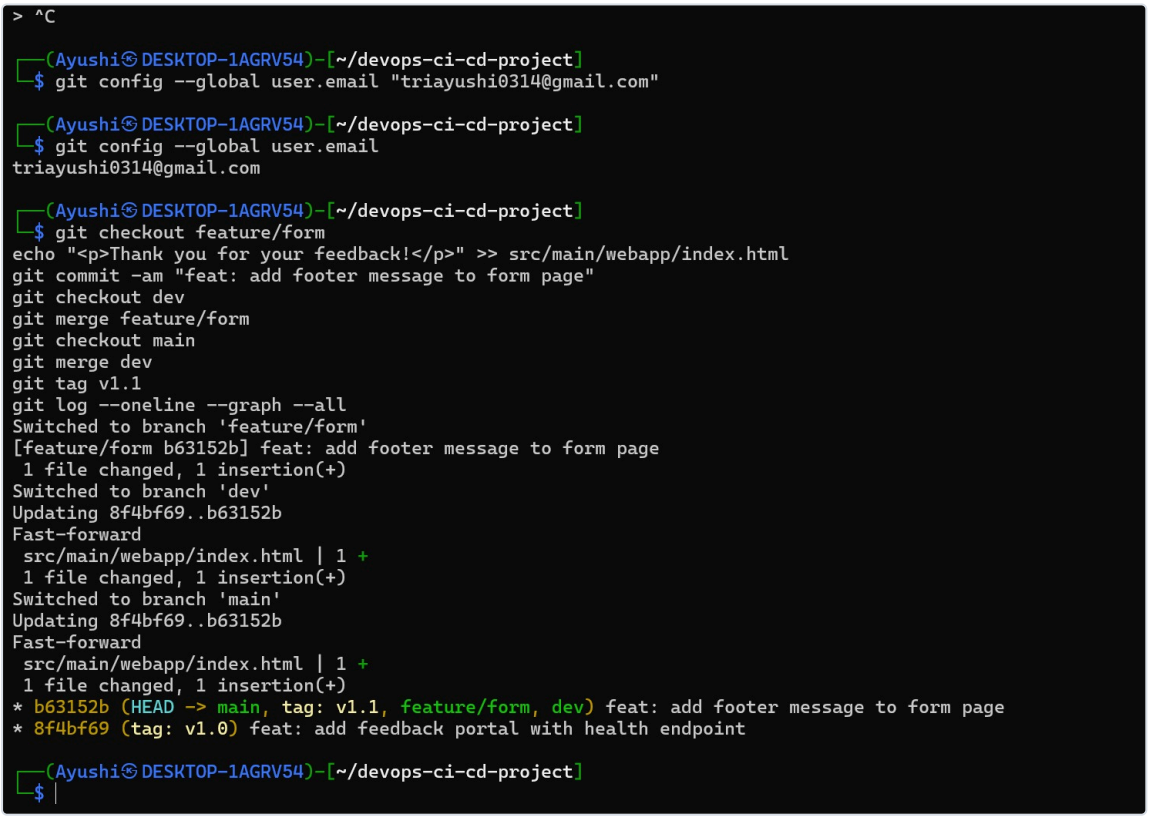


Figure 3: Feature commit, merges into dev and main, and tag v1.1 in the commit graph.

6.4 GitHub remote repository

The project is hosted on GitHub on branch `main` with the folders and files `src/`, `.gitignore`, `Jenkinsfile` and `pom.xml`. The latest commit shown is `ci: fix health check port to 8081`.

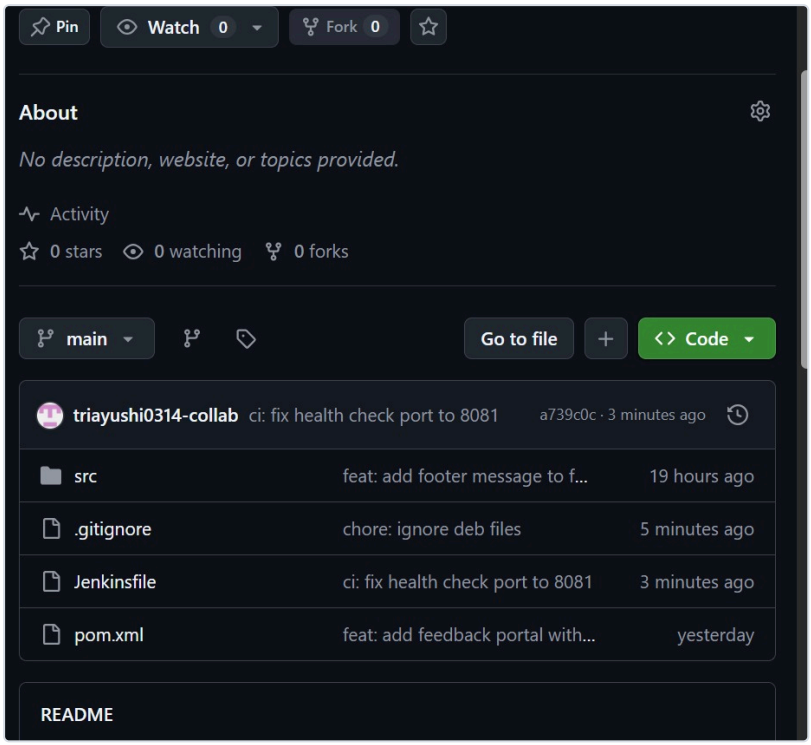


Figure 4: GitHub repository on branch main with commit history.

6.5 Maven project and first local build

The `pom.xml` declares the project as `com.demo:feedback-portal`, version 1.0, with packaging `war` and the final artifact name `app`, so the build always produces `target/app.war`. The first local build downloaded all Maven plugins from Maven Central and took about 37 seconds; later builds are much faster because the plugins are cached.

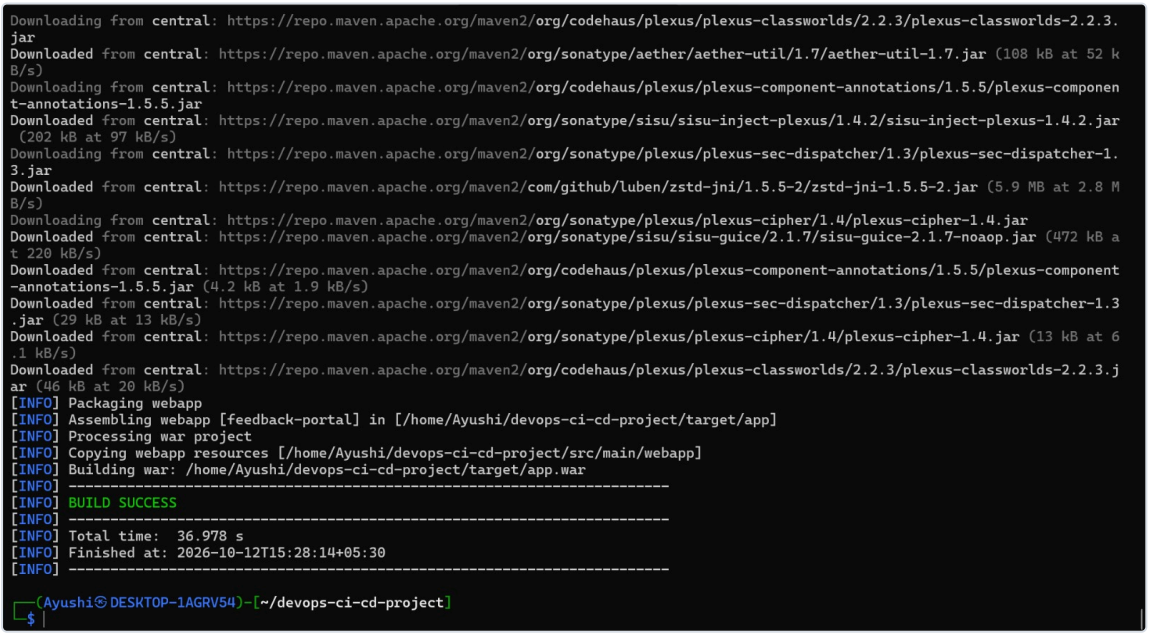


Figure 5: Local Maven build: BUILD SUCCESS and WAR generated at target/app.war (first build, plugins downloaded).

6.6 Apache Tomcat installation

Tomcat 10.1.34 was installed in `/opt/tomcat` and started with `sudo /opt/tomcat/bin/startup.sh`. Startup output confirms `CATALINA_BASE` and `CATALINA_HOME` as `/opt/tomcat` and `JRE_HOME` as `/usr`. The connector port was set to 8081 so that it does not clash with Jenkins.

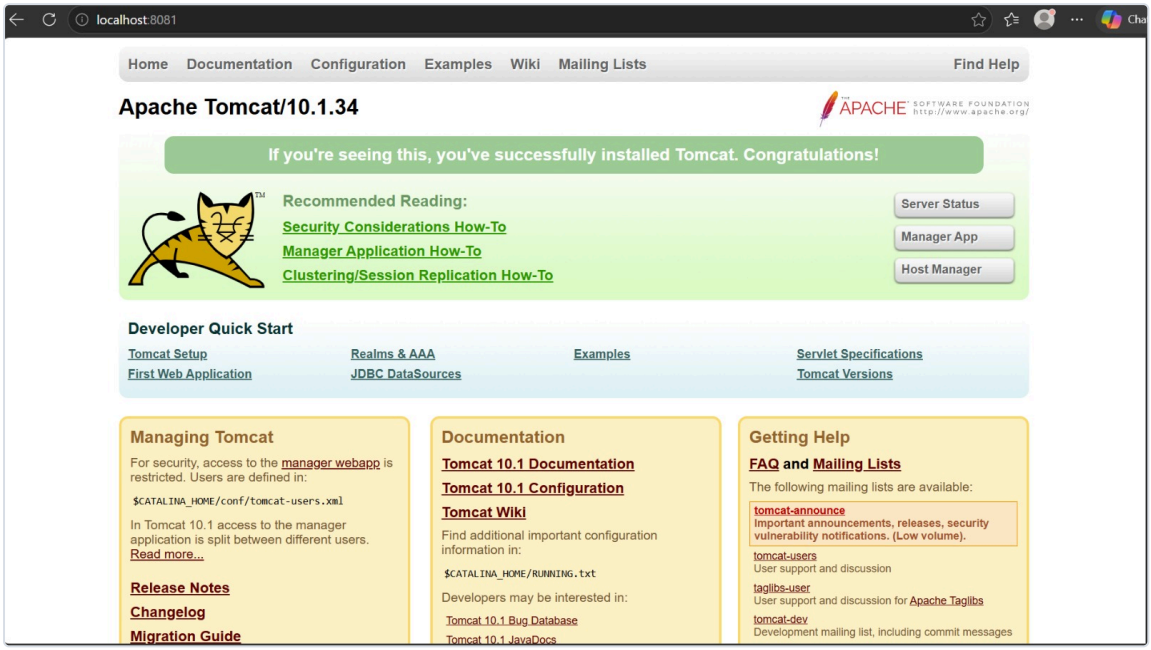


Figure 6: Tomcat 10.1.34 default page confirming a working installation on localhost:8081.

6.7 Jenkins installation and setup

Jenkins was started on port 8085. On first access the suggested plugins were installed, the setup wizard was completed and the dashboard became available with 2 build executors.

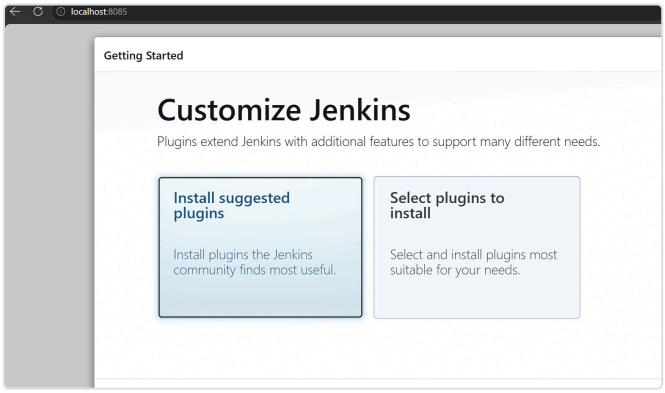


Figure 7: Jenkins first-time setup: "Install suggested plugins".

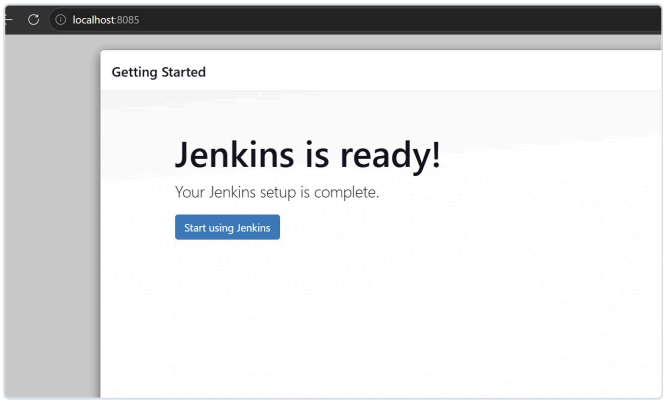


Figure 8: "Jenkins is ready!" The setup is complete.

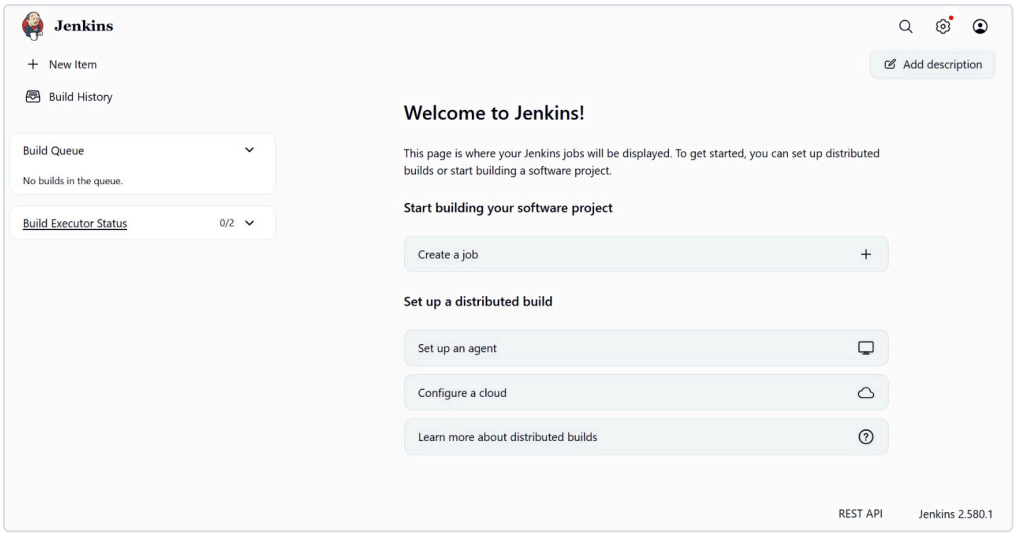


Figure 9: Jenkins dashboard (version 2.580.1) with an empty build queue and 2 executors.

6.8 Jenkins job configuration

The job `devops-ci-cd-pipeline` executes one shell build step. The commands visible in its console log are:

Step	Command	Purpose
1	`rm -rf /tmp/proj`	Remove the previous copy so every build starts clean.
2	`git clone <repository> /tmp/proj`	Checkout the latest code from the GitHub repository.
3	`cd /tmp/proj`	Move into the project directory.
4	`mvn clean package`	Compile, run unit tests and build `target/app.war`.
5	`cp target/app.war /opt/tomcat/webapps/`	Deploy the WAR to Tomcat. Runs only if Maven succeeded.

The shell is started with `-xe` (visible as `/bin/sh -xe` in the log): `-x` prints every command and `-e` stops the script at the first failing command, so the job stops at the first failing command. This is what makes the test failure a real gate: if Maven fails, the copy step is never reached.

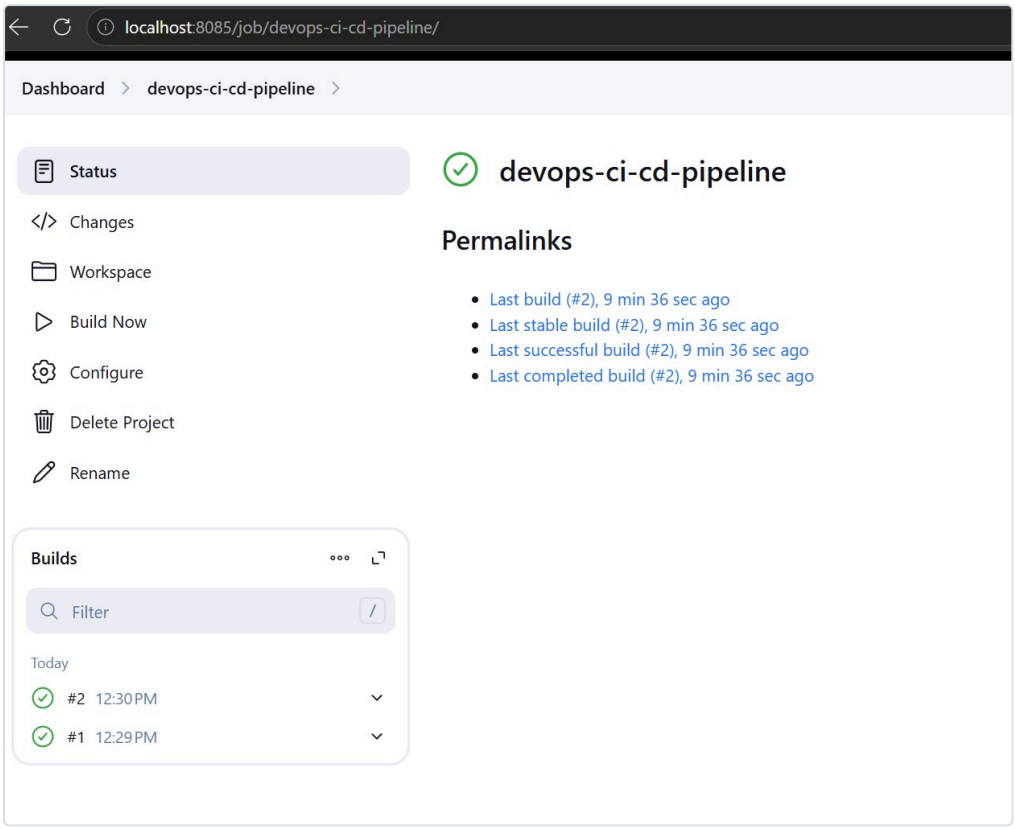


Figure 10: Jenkins job devops-ci-cd-pipeline with successful builds #1 and #2 on 13 October.

6.9 Build #6: console walk-through

Console line	Meaning
Started by user Ayushi tripathi	Build was triggered manually by the logged-in user.
`git clone ... /tmp/proj`	Latest source fetched from GitHub.
Compiling 3 source files ... target 21	FeedbackServlet, HealthServlet and Status compiled.
Running com.demo.StatusTest, Tests run: 1, Failures: 0	The unit test passed, so the pipeline continues.
Building war: /tmp/proj/target/app.war	WAR artifact created.
BUILD SUCCESS, Total time: 3.941 s	Build finished at 2026-10-13 22:48:46 (+05:30).
`cp target/app.war /opt/tomcat/webapps/`	Deployment step executed.
Finished: SUCCESS	Jenkins marks the build green.

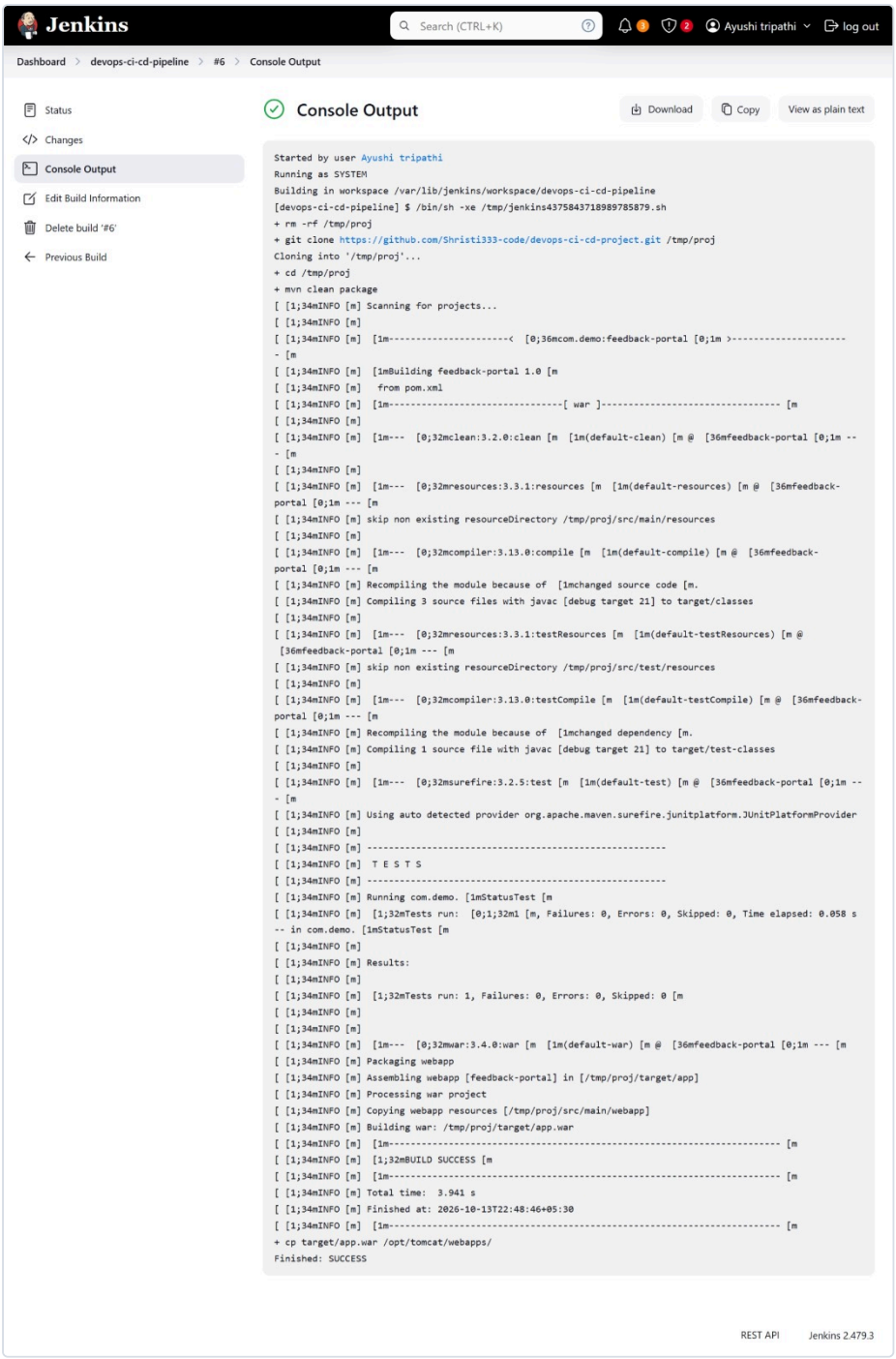


Figure 11: Jenkins console output of build #6: clone, mvn clean package, tests passed, WAR copied to Tomcat, Finished: SUCCESS.

6.10 Build #7: repeat run

Build #7 was run again to confirm the pipeline is repeatable. The tests passed again (Tests run: 1, Failures: 0), `app.war` was rebuilt in 3.895 seconds at 22:51:53 and copied to Tomcat.

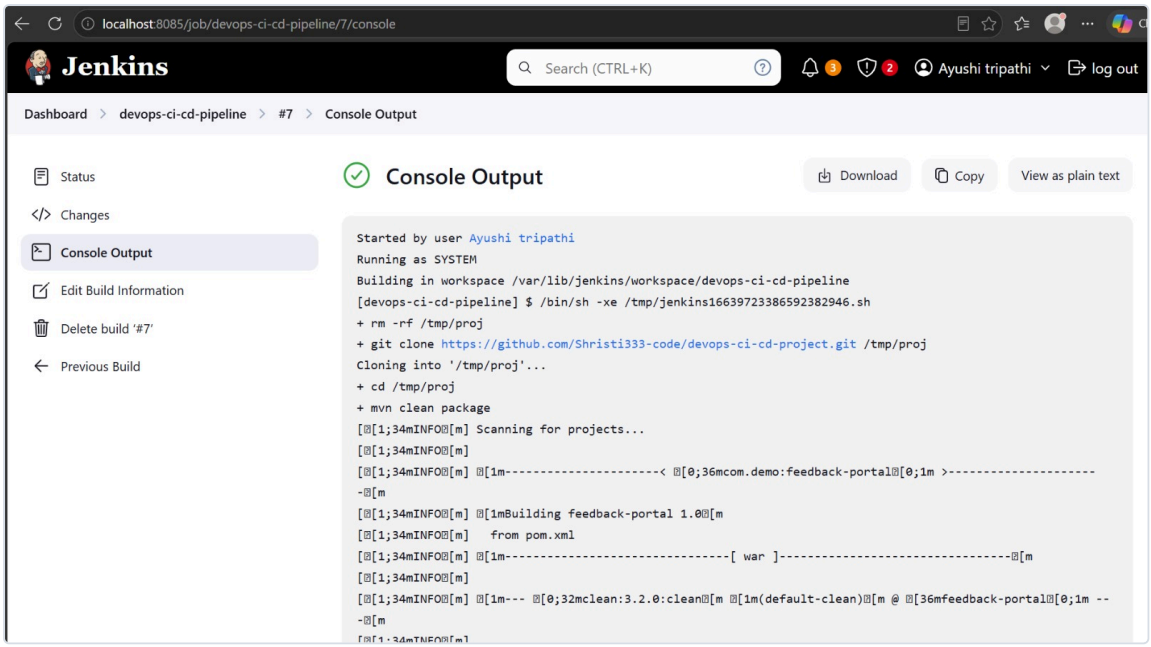


Figure 12: Build #7 console (start): triggered by Ayushi, repository cloned, `mvn clean package` started.

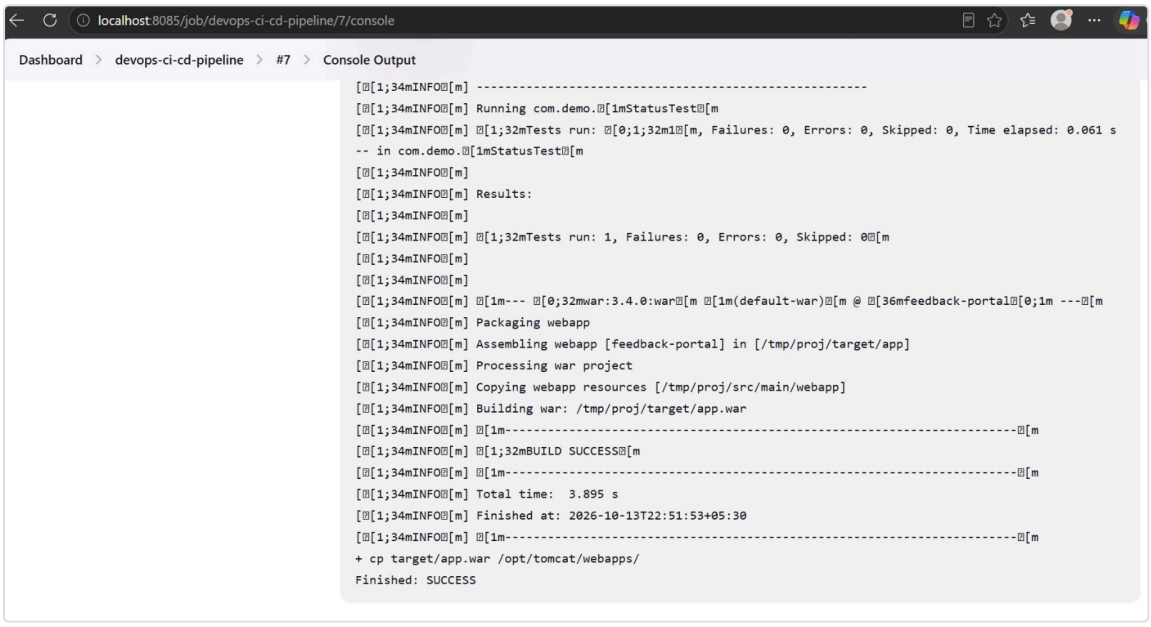


Figure 13: Build #7 console (end): tests passed, Building war: /tmp/proj/target/app.war, `cp` to Tomcat, Finished: SUCCESS.

7. Deployment Verification

After the pipeline copied `app.war` into Tomcat, the application was opened at `http://localhost:8081/app/`. The checks below were performed.

7.1 Home page

The heading shows **"Student Feedback Portal: BY Ayushi"**, followed by the Name, Email and Feedback fields, the Submit button, a Health check link and the footer line from the latest commit.

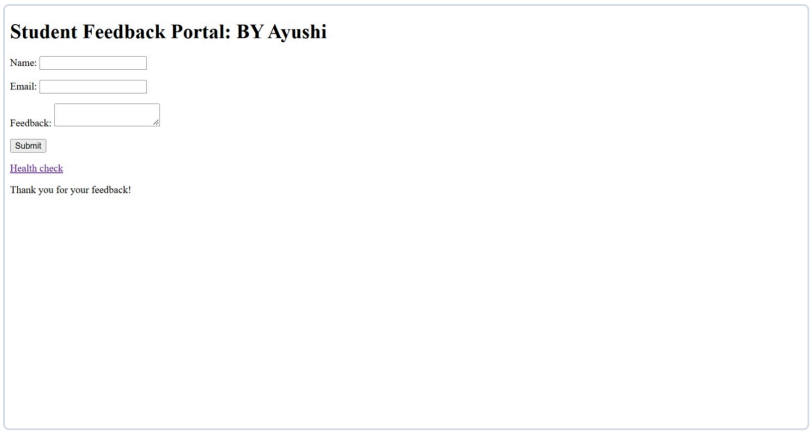


Figure 14: Deployed application: updated heading, form, Health check link and footer message.

7.2 Form submission

The form was filled with test data and submitted. The confirmation page repeated the entered name, email and feedback, which proves that the servlet is deployed and working.

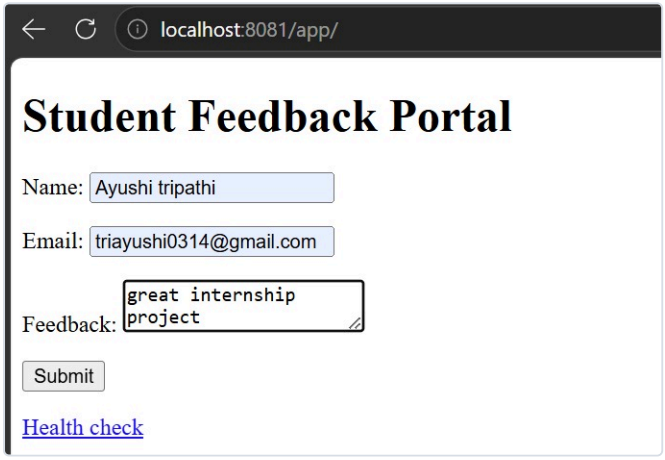


Figure 15: Feedback form filled with test data at /app/.

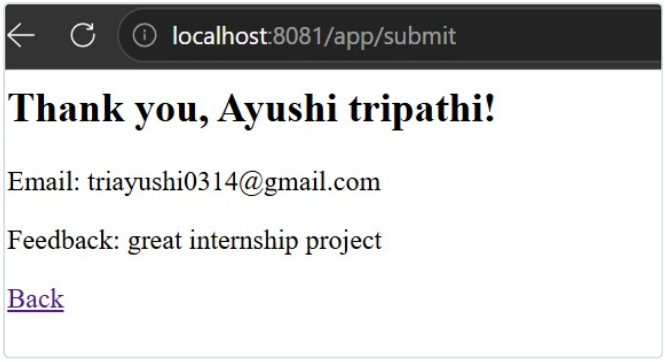


Figure 16: Confirmation page at /app/submit after submission.

7.3 Health endpoint

The endpoint `/app/health` returns the text `OK` with HTTP 200. This is the signal used by the pipeline to decide whether a deployment is healthy. It was checked in the browser at several points: on the first deployment and again after the Jenkins runs.

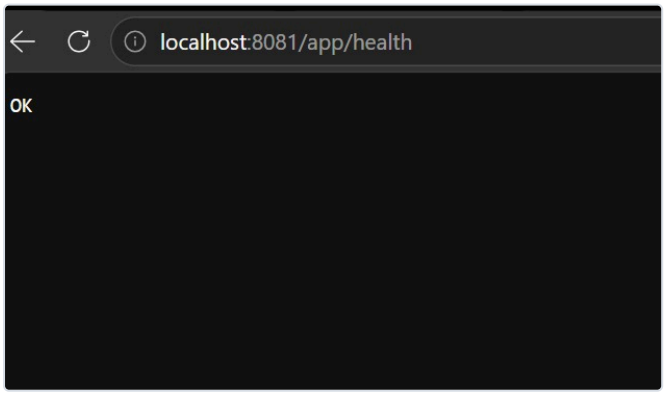


Figure 17: Health endpoint after the first deployment: OK.

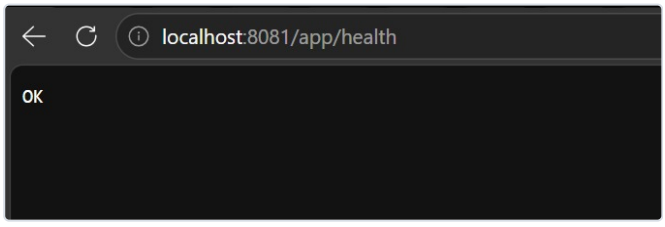


Figure 18: Health endpoint after the Jenkins-deployed build: OK.

8. Testing and Test Cases

ID	Test case	Expected result	Actual result	Status
TC1	Open `/app/`	Form page with heading loads	Page loaded with heading and form	**Pass**
TC2	Submit the form	Confirmation shows submitted data	Thank-you page showed name, email, feedback	**Pass**
TC3	Open `/app/health`	HTTP 200 and body OK	HTTP 200, body OK	**Pass**
TC4	Unit test `StatusTest` in Jenkins	Tests run: 1, Failures: 0	Tests run: 1, Failures: 0 (builds #6, #7)	**Pass**
TC5	Jenkins build #6 and #7	BUILD SUCCESS, WAR copied to Tomcat	Finished: SUCCESS	**Pass**
TC6	Deliberately failing test	Build fails, no deployment	BUILD FAILURE, Failures: 1	**Pass (gate works)**
TC7	Deploy a corrupt WAR	Failure detected by health check	HTTP 404 on /app/health	**Pass (detected)**
TC8	Restore backup WAR	Health check returns HTTP 200	HTTP 200, body OK	**Pass**

9. Failure Handling and Rollback

9.1 Case A: failing unit test stops the pipeline

The unit test `StatusTest.healthMessageIsOk` was deliberately edited so that it expected `FAIL` instead of `OK`. Maven reported:

```
[ERROR] Failures:
[ERROR]   StatusTest.healthMessageIsOk:9 expected: <FAIL> but was: <OK>
[ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0
[INFO] BUILD FAILURE
```

Because the build failed in the test phase, `app.war` was never produced and nothing was deployed. Maven also pointed to `target/surefire-reports` for the individual test results. The test directory was then restored from Git with `git checkout -- src/test`, which returned the repository to the clean state of the `main` branch.

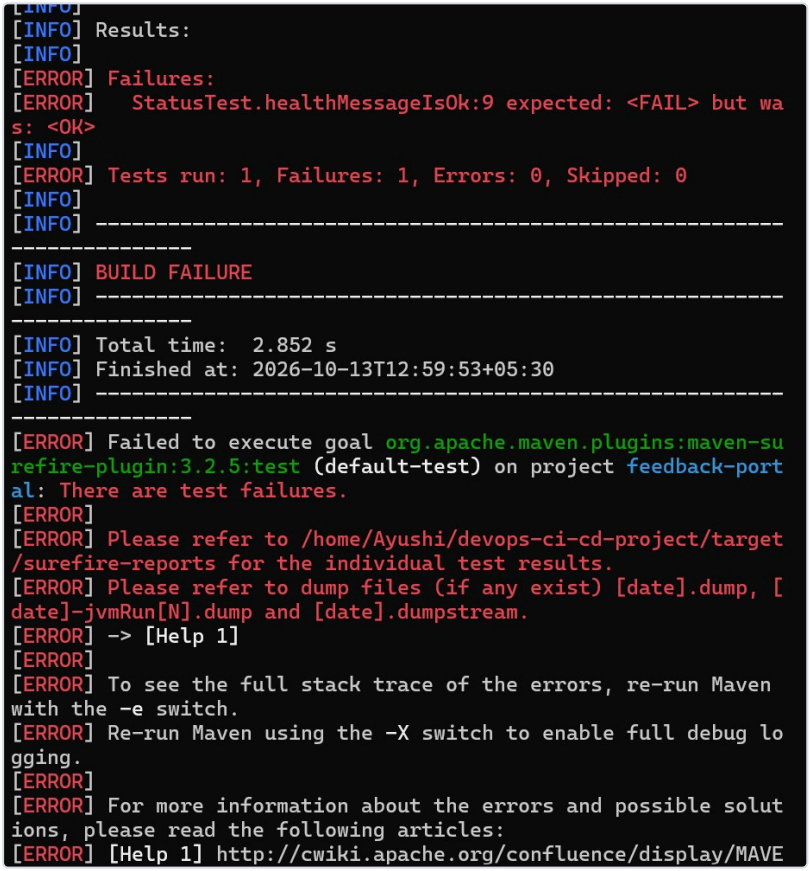


Figure 19: StatusTest failed (expected FAIL but was OK): Tests run: 1, Failures: 1, BUILD FAILURE.

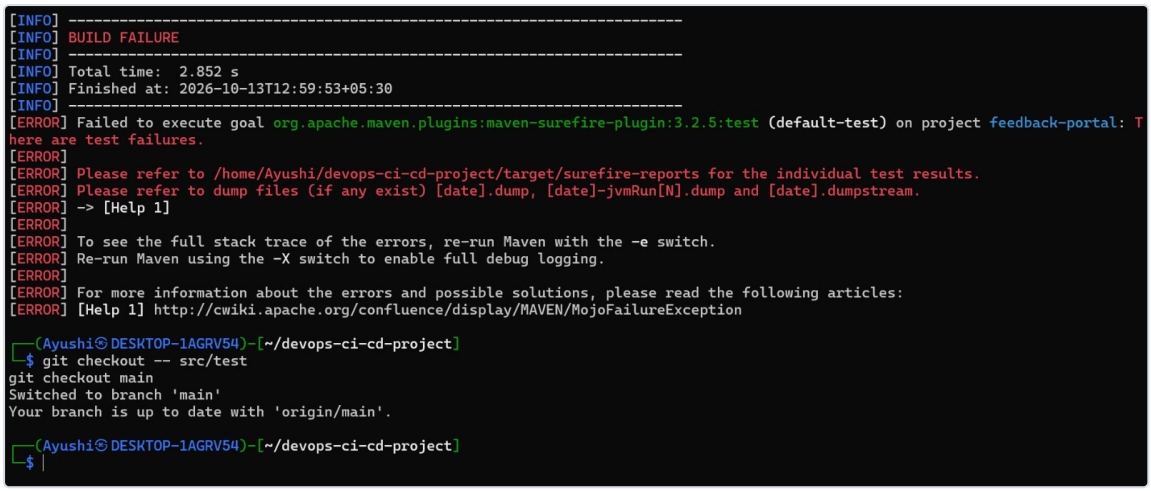


Figure 20: Maven surefire failure, followed by `git checkout -- src/test` restoring the test (branch main, up to date with origin/main).

9.2 Case B: bad deployment detected by the health check

Before the experiment, the working WAR was saved as a backup:

```
cp /opt/tomcat/webapps/app.war ~/app-v1.0-backup.war
ls -l ~/app-v1.0-backup.war        # 4402 bytes
```

A corrupt file was then deployed in place of the real WAR:

```
echo "broken" > /tmp/bad.war
cp /tmp/bad.war /opt/tomcat/webapps/app.war
sleep 10
curl -i http://localhost:8081/app/health
```

The health check immediately exposed the problem: Tomcat answered **HTTP/1.1 404 Not Found** ("The requested resource [/app/health] is not available") because the corrupt archive could not be deployed. A green pipeline without this check would have hidden the failure, which is why the health check is mandatory.

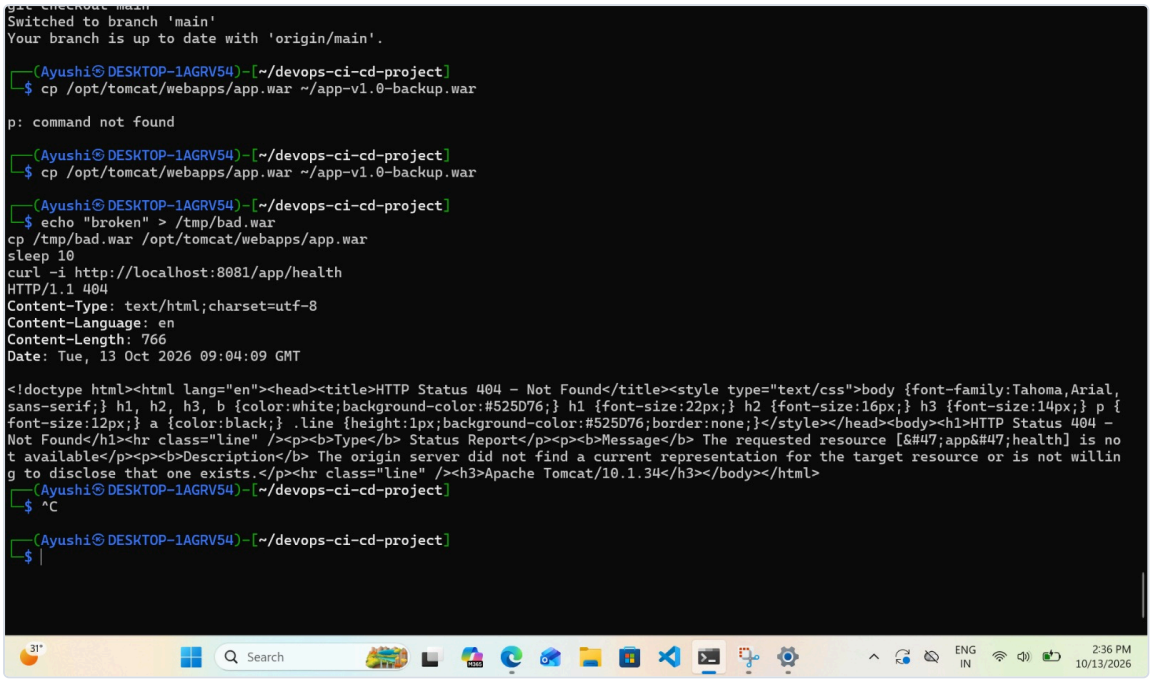


Figure 21: Backup created, broken WAR deployed, health check returns HTTP 404 Not Found.

9.3 Rollback procedure

The previous good version was restored from the backup:

1. Make sure Tomcat is running: `sudo /opt/tomcat/bin/startup.sh`.
2. Remove the broken deployment: `rm -rf /opt/tomcat/webapps/app` (this was denied without sudo because of directory permissions, but did not block the recovery).
3. Copy the backup over the deployed WAR: `cp ~/app-v1.0-backup.war /opt/tomcat/webapps/app.war`.
4. Wait for Tomcat to redeploy: `sleep 20`.
5. Verify: `curl -i http://localhost:8081/app/health`.

The check returned **HTTP/1.1 200** with body `OK` (Tue, 13 Oct 2026 09:14:29 GMT), proving the rollback succeeded.

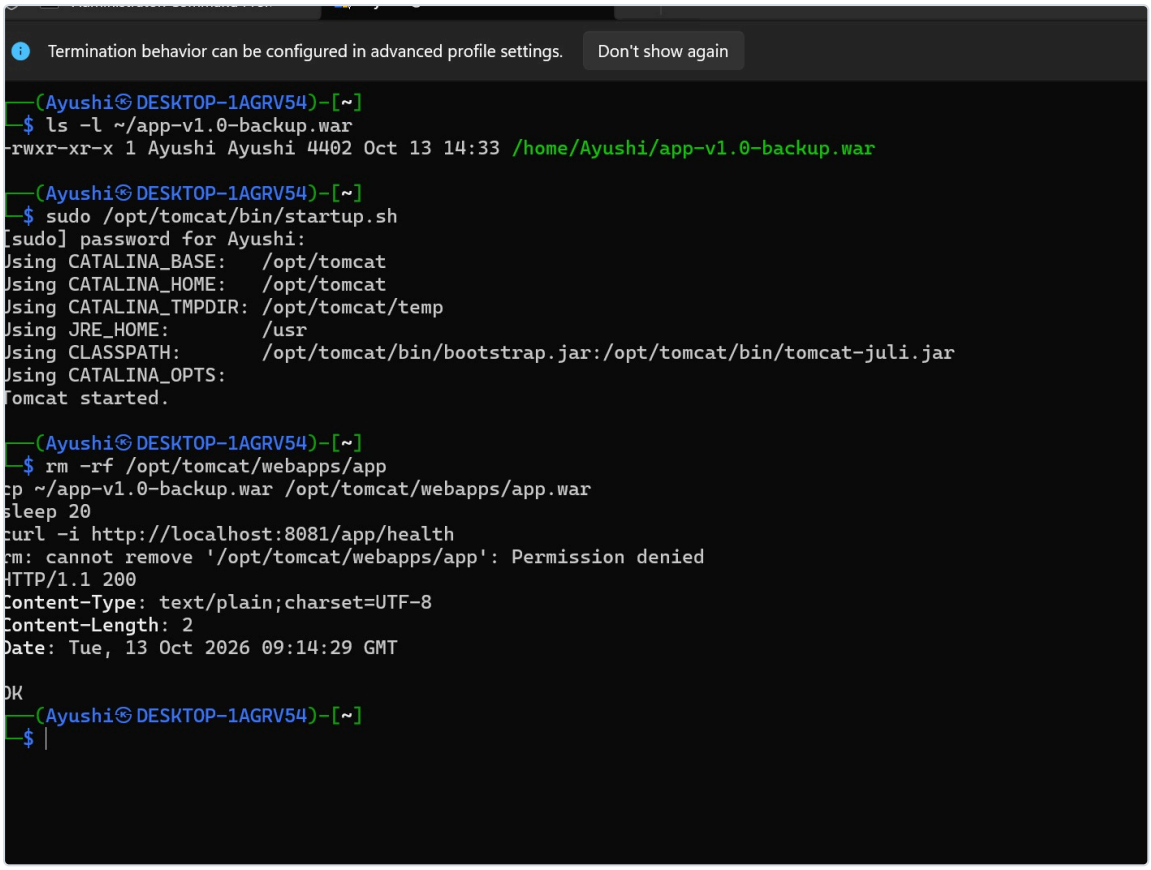


Figure 22: Backup verified, Tomcat started, backup WAR restored: /app/health returns HTTP 200 and body OK.

9.4 Recovery options available

Situation	Recovery
Broken test or code	Pipeline stops at the Maven test phase; fix the code or restore with `git checkout -- <path>` and run the job again.
Bad WAR in Tomcat	Copy the last good WAR (backup) back to `webapps/app.war` and verify with `curl`.
Need an older release	Check out tag `v1.0` or `v1.1` and rebuild with `mvn clean package`.

10. Challenges and Solutions

Challenge	Solution
Port conflict between Jenkins and Tomcat (both default to 8080)	Jenkins runs on 8085 and Tomcat on 8081. The health-check port in the repository was updated with `ci: fix health check port to 8081`.
Permission denied on `/opt/tomcat/webapps` (root-owned directory)	Write permission on the webapps directory was given to the Jenkins user so the deploy step `cp target/app.war /opt/tomcat/webapps/` works. Administrative actions were run with `sudo`.

Application returned 404 after a bad deployment	Detected through the health check and fixed by restoring the previous good WAR kept as a backup.
Broken test must not reach production	The Maven test phase acts as a quality gate: BUILD FAILURE stops the job before the copy step.
Slow first build	The first build took about 37 seconds because Maven downloaded its plugins. Later builds finished in about 4 seconds from the local cache.

11. Risks and Mitigation

Risk	Impact	Mitigation applied
Port conflict (Jenkins vs Tomcat)	High	Separate ports 8085 and 8081.
Flaky or missing Maven dependencies	Medium	First build verified locally; plugins cached afterwards.
Permission errors on deploy	Medium	Correct directory permissions for the Jenkins user; sudo for admin steps.
Pipeline succeeds but application is broken	High	Mandatory health-check verification after deployment.
Previous good build lost	Medium	Backup WAR kept and Git tags `v1.0`, `v1.1` mark known-good releases.

12. Results: Manual vs Automated Delivery

Aspect	Manual process	With the CI/CD pipeline
Build	Run Maven by hand on the developer machine	Jenkins runs `mvn clean package` in about 4 seconds
Testing	Easy to skip	Unit test runs on every build and blocks bad code
Deployment	Copy WAR to Tomcat manually	Automatic `cp` to webapps after a successful build
Verification	Open the browser and hope	Defined health check (HTTP 200, body OK)
Traceability	No record of what was deployed	Git history, tags and numbered Jenkins builds with full logs
Recovery	No plan	Backup WAR and documented rollback steps

Outcome: a code change reaches the running application with one Jenkins build, and any failure is visible in the logs or the health check.

13. Limitations and Future Improvements

- **Pipeline as code:** the repository contains a `Jenkinsfile`; the job can be moved to a declarative pipeline with separate Checkout, Build, Test, Package, Archive, Deploy and Verify stages for clearer stage-wise logs.
- **Automatic trigger:** builds were started manually from Jenkins; a GitHub webhook or SCM polling would start them on every push.
- **Automated health check and rollback:** the health check and rollback were run manually; they can be added as a final pipeline stage that restores the last good WAR if `curl` does not return 200.
- **Artifact archiving:** archiving each WAR in Jenkins would give a ready backup for every build number.
- **Credentials:** any repository or server credentials should be kept in the Jenkins credentials store instead of the job script.
- **Richer tests:** only one unit test exists; adding tests for the servlets would make the quality gate stronger.

14. Deliverables Checklist

Deliverable	Evidence in this report
Git repository with commit history, branches and tags	Sections 6.1 - 6.4
Working Maven project (WAR packaging)	Sections 5.3, 6.5
Jenkins job and pipeline runs	Sections 6.7 - 6.10
Successful and failed build logs	Sections 6.9, 6.10, 9.1
Generated WAR artifact (app.war)	Sections 6.5, 6.9, 6.10
Tomcat deployment proof	Sections 6.6, 7
Health-check output	Sections 7.3, 9.3
Rollback demonstration	Section 9
Dataset notes	Section 2.2 (dataset not required)

15. Conclusion

This project delivered a working, automated path from a Git commit to a running web application on Apache Tomcat. Jenkins clones the latest code, Maven compiles, tests and packages it, the

generated `app.war` is deployed automatically, and the application is verified through `/app/health`. The failure demonstrations proved two important properties: a broken test is stopped before deployment, and a bad deployment can be detected quickly and rolled back to the previous good WAR. Compared with manual delivery the process is faster, repeatable, traceable and safer, and it provides a solid base for the improvements listed in Section 13.